REPORTE DE PRÁCTICA NO. 3

Construcción de un Autómata Finito Determinista

ALUMNO: Ángel Ortega Gómez
Dr. Eduardo Cornejo Velázquez

1. Introducción

En el área de la teoría de la computación, los autómatas finitos representan uno de los modelos más fundamentales para comprender cómo una máquina puede procesar información de manera estructurada. Estos modelos matemáticos permiten describir sistemas que reciben una secuencia de símbolos como entrada y, a partir de un conjunto definido de estados y reglas de transición, determinan si dicha secuencia pertenece o no a un lenguaje específico. Debido a su simplicidad y claridad conceptual, los autómatas finitos se utilizan ampliamente para explicar el funcionamiento de diversos procesos computacionales relacionados con el reconocimiento de patrones, el análisis de cadenas y el diseño de compiladores.

Dentro de esta categoría, el Autómata Finito Determinista (AFD) se caracteriza porque para cada estado y cada símbolo del alfabeto existe exactamente una transición posible hacia otro estado. Esta propiedad garantiza que el comportamiento del autómata esté completamente definido para cualquier cadena de entrada, lo que facilita su implementación en programas de computadora. A partir de un estado inicial, el autómata recorre diferentes estados de acuerdo con los símbolos que va leyendo, hasta llegar a un estado final que determina si la cadena es aceptada o rechazada.

La implementación de un AFD en un lenguaje de programación permite trasladar este modelo teórico a un entorno práctico, donde es posible evaluar múltiples cadenas de símbolos y verificar si cumplen con las reglas del lenguaje definido por el autómata. De esta manera, se puede observar cómo los conceptos formales de estados, transiciones y alfabetos se traducen en estructuras de datos y procedimientos dentro de un programa.

En esta práctica se desarrolla un programa en un lenguaje de programación de alto nivel que implementa un Autómata Finito Determinista capaz de analizar diferentes cadenas de entrada. El sistema recibe la descripción formal del autómata, incluyendo el conjunto de estados, el alfabeto, las transiciones, el estado inicial y los estados de aceptación, para posteriormente evaluar un conjunto de cadenas y determinar si son aceptadas o rechazadas por el autómata. Con ello se busca reforzar la comprensión de los elementos que componen un AFD y su funcionamiento dentro del reconocimiento de lenguajes formales.

2. Marco teórico

Lenguajes formales

En el estudio de la teoría de la computación, los lenguajes formales constituyen una herramienta fundamental para describir y analizar la estructura de diferentes tipos de cadenas de símbolos. Un lenguaje formal puede definirse como un conjunto de cadenas construidas a partir de un alfabeto finito de símbolos. Estas cadenas siguen ciertas reglas o patrones que permiten determinar si pertenecen o no al lenguaje en cuestión. El análisis de estos lenguajes permite comprender cómo las computadoras procesan información y cómo se pueden diseñar sistemas capaces de reconocer estructuras específicas dentro de una secuencia de datos [2].

Los lenguajes formales son ampliamente utilizados en diversas áreas de la informática, particularmente en el diseño de compiladores, el análisis sintáctico y el reconocimiento de patrones. Para estudiar estos lenguajes se emplean diferentes modelos matemáticos capaces de describir su comportamiento, entre los cuales destacan los autómatas finitos, las gramáticas formales y las máquinas de Turing. Cada uno de estos modelos permite analizar diferentes clases de lenguajes y comprender sus propiedades computacionales [1].

Autómatas finitos

Los autómatas finitos son modelos matemáticos utilizados para representar sistemas que pueden encontrarse en uno de varios estados posibles y que cambian de estado en respuesta a símbolos de entrada. Este tipo de modelo se caracteriza por tener un número finito de estados y por procesar las entradas de manera secuencial. A medida que el autómata recibe símbolos del alfabeto, utiliza una función de transición para determinar el siguiente estado al que debe dirigirse [3].

Formalmente, un autómata finito está compuesto por un conjunto de estados, un alfabeto de entrada, una función de transición, un estado inicial y un conjunto de estados de aceptación. Durante su operación, el autómata comienza en el estado inicial y procesa cada símbolo de la cadena de entrada. Una vez que se ha leído toda la cadena, el sistema determina si esta es aceptada o rechazada dependiendo del estado final en el que se encuentre. Si el estado final pertenece al conjunto de estados de aceptación, entonces la cadena es considerada válida dentro del lenguaje reconocido por el autómata [1].

Este modelo teórico resulta especialmente útil debido a su simplicidad y a su capacidad para describir sistemas de reconocimiento de patrones relativamente sencillos. En la práctica, los autómatas finitos se utilizan en aplicaciones como el análisis léxico en compiladores, los sistemas de validación de datos y el diseño de protocolos de comunicación [4].

Autómata Finito Determinista (AFD)

Dentro de los diferentes tipos de autómatas finitos, uno de los más importantes es el Autómata Finito Determinista (AFD). Este tipo de autómata se caracteriza porque, para cada combinación posible entre un estado y un símbolo del alfabeto, existe exactamente una transición definida hacia otro estado. Esto significa que el comportamiento del autómata está completamente determinado por su estado actual y por el símbolo de entrada que se está procesando [2].

Formalmente, un AFD se define como una quíntupla:

$$A = (Q, \Sigma, \delta, q_0, F) \tag{1}$$

donde:

- Q es un conjunto finito de estados.
- Σ es un alfabeto finito de entrada.
- $\delta : Q \times \Sigma \to Q$ es la función de transición.
- $q_0 \in Q$ es el stado inicial.
- $F \subseteq Q$ es el conjunto de estados de aceptación.

La función de transición indica que para cada estado y cada símbolo del alfabeto existe una transición hacia un único estado siguiente [3].

Función de transición y procesamiento de cadenas

La función de transición es el mecanismo que permite al autómata cambiar de estado cuando recibe un símbolo de entrada. Esta función establece una relación entre el estado actual del sistema y el símbolo leído, determinando cuál será el siguiente estado del autómata. De esta manera, el procesamiento de una cadena se realiza de forma secuencial, símbolo por símbolo, siguiendo las transiciones definidas dentro del autómata [1].

Si se considera una cadena $w = w_1 w_2 \dots w_n$, donde cada símbolo $w_i \in \Sigma$, el autómata comienza en el estado inicial y aplica sucesivamente la función de transición para cada símbolo de la cadena. El resultado final será el estado alcanzado después de procesar todos los símbolos de la entrada.[1].

Lenguaje reconocido por un AFD

Un Autómata Finito Determinista reconoce un lenguaje formado por todas las cadenas que, al ser procesadas por el autómata, terminan en un estado de aceptación. Es decir, el lenguaje reconocido por un AFD corresponde al conjunto de cadenas que cumplen con las reglas definidas por la función de transición y los estados del sistema [2].

Formalmente, si se tiene un autómata A, el lenguaje reconocido por dicho autómata se define como el conjunto de todas las cadenas que son aceptadas por él. Este concepto es fundamental dentro de la teoría de los lenguajes formales, ya que permite establecer una relación directa entre los modelos de computación y las estructuras de los lenguajes que estos son capaces de reconocer. Gracias a esta relación es posible analizar diferentes problemas computacionales y desarrollar métodos eficientes para el reconocimiento de patrones dentro de cadenas de símbolos [5].

3. Obejtivos

3.1 Objetivo general

Desarrollar un programa en un lenguaje de programación de alto nivel que implemente un Autómata Finito Determinista (AFD), capaz de analizar diferentes cadenas de símbolos y determinar si pertenecen o no al lenguaje reconocido por el autómata, a partir de la definición de sus estados, alfabeto, función de transición, estado inicial y estados de aceptación.

3.2 Objetivos específicos

- Identificar los componentes fundamentales que conforman un Autómata Finito Determinista, incluyendo el conjunto de estados, el alfabeto de entrada, la función de transición, el estado inicial y los estados de aceptación.
- Representar de manera computacional la estructura de un AFD mediante estructuras de datos adecuadas dentro de un lenguaje de programación de alto nivel.
- Implementar el mecanismo de transición entre estados que permita procesar cada símbolo de una cadena de entrada siguiendo las reglas definidas por el autómata.
- Analizar diferentes cadenas de símbolos para determinar si son aceptadas o rechazadas por el autómata, verificando si el estado final alcanzado pertenece al conjunto de estados de aceptación.
- Evaluar el funcionamiento del programa mediante la prueba de múltiples cadenas de entrada, comprobando que el sistema identifique correctamente aquellas que pertenecen al lenguaje reconocido por el AFD.

4. Herramientas empleadas

Para el desarrollo de la práctica se emplearon distintas herramientas de software que permitieron tanto la implementación del programa como la verificación del funcionamiento del autómata. Estas herramientas facilitaron el proceso de programación, compilación y prueba del Autómata Finito Determinista desarrollado.

1. Java.Lenguaje de programación de alto nivel, orientado a objetos y multiplataforma, utilizado para la implementación de los algoritmos de análisis de cadenas y el uso de expresiones regulares. Java ofrece bibliotecas estándar robustas para el manejo de texto y validación de patrones, lo que lo hace adecuado para el desarrollo de aplicaciones de análisis léxico.

2. Apache NetBeans IDE 17. Entorno de desarrollo integrado utilizado para la escritura, compilación y ejecución de los programas desarrollados en Java. NetBeans proporciona herramientas de depuración y organización del código que facilitan el desarrollo estructurado y la detección de errores durante la implementación.

3. Java Development Kit (JDK) 17. Conjunto de herramientas necesarias para el desarrollo de aplicaciones en Java, el cual incluye el compilador, la máquina virtual y las bibliotecas estándar del lenguaje. Esta versión garantiza compatibilidad con las características modernas del lenguaje utilizadas en la práctica.

4. Automaton Simulator. Plataforma web, la cual permitió representar gráficamente el autómata y verificar el comportamiento de las transiciones entre estados, lo que resultó útil para comprobar que la lógica implementada en el programa coincidiera con el funcionamiento esperado del modelo teórico.

5. Desarrollo

En esta práctica se desarrolló un programa capaz de implementar el funcionamiento de un Autómata Finito Determinista (AFD) mediante el uso de un lenguaje de programación de alto nivel. El objetivo principal del sistema fue analizar diferentes cadenas de símbolos para determinar si estas pertenecen o no al lenguaje reconocido por el autómata definido. Para lograrlo, el programa recibe como entrada la descripción completa del autómata, incluyendo el conjunto de estados, el alfabeto, las funciones de transición, el estado inicial y los estados de aceptación.

El programa fue desarrollado en Java, utilizando estructuras de datos básicas como arreglos y variables primitivas para representar los distintos elementos que componen el autómata. A partir de estos elementos se implementó un procedimiento que permite simular el comportamiento del AFD al procesar cadenas de símbolos.

5.1 Lectura del archivo de entrada

El funcionamiento del programa comienza con la lectura de un archivo de texto denominado afd.txt. Este archivo contiene toda la información necesaria para definir el autómata y las cadenas que deben ser evaluadas. Para realizar esta tarea se utiliza la clase *Scanner*, que permite leer datos desde archivos de manera secuencial.

En la primera línea del archivo se encuentran seis valores enteros que describen las características generales del autómata. Estos valores corresponden al número de estados del sistema, el tamaño del alfabeto, la cantidad de funciones de transición, el estado inicial, el número de estados de aceptación y la cantidad de cadenas que deben ser analizadas. Cada uno de estos parámetros es almacenado en variables independientes que posteriormente serán utilizadas para construir la estructura interna del autómata.

Durante esta etapa también se realiza una validación básica de los parámetros de entrada. El programa verifica que los valores se encuentren dentro de los rangos permitidos establecidos en la especificación del problema. En caso de que alguno de los valores sea inválido, el programa muestra un mensaje de error y finaliza su ejecución. Este mecanismo permite evitar inconsistencias que puedan afectar el funcionamiento del autómata.

5.2 Definición del alfabeto

Después de leer los parámetros iniciales, el programa procede a leer el alfabeto del autómata. El alfabeto representa el conjunto de símbolos que pueden aparecer dentro de las cadenas de entrada. Para almacenar estos símbolos se utiliza un arreglo de caracteres cuyo tamaño corresponde al número de símbolos especificado previamente.

Cada símbolo es leído desde el archivo de entrada y almacenado en una posición del arreglo. Este arreglo permite posteriormente verificar si los símbolos presentes en las cadenas pertenecen realmente al alfabeto definido por el autómata. De esta manera se evita que el programa procese caracteres que no forman parte del lenguaje especificado.

5.3 Estados de aceptación

Una vez definido el alfabeto, el programa procede a leer los estados de aceptación del autómata. Estos estados representan las configuraciones finales en las que el sistema puede encontrarse después de procesar

una cadena. Si al terminar la lectura de una cadena el autómata se encuentra en alguno de estos estados, la cadena es considerada válida dentro del lenguaje reconocido.

Para representar esta información se utiliza un arreglo de tipo booleano. Cada posición del arreglo corresponde a un estado del autómata y permite indicar si dicho estado es o no un estado de aceptación. El tamaño del arreglo es $N+1$, lo que permite utilizar directamente los números de estado como índices dentro del arreglo, evitando la necesidad de realizar conversiones adicionales.

Durante esta etapa también se verifica que los estados indicados como finales se encuentren dentro del rango permitido. Si se detecta un estado fuera del conjunto definido, el programa genera un mensaje de error.

5.4 Definición de las funciones de transición

Uno de los componentes más importantes del autómata es la función de transición, ya que esta define cómo cambia el estado del sistema cuando se procesa un símbolo de entrada. En el programa, las transiciones se almacenan utilizando una matriz bidimensional de enteros.

Las filas de la matriz representan los estados del autómata, mientras que las columnas corresponden a los símbolos del alfabeto. De esta forma, cada posición de la matriz almacena el estado al que debe dirigirse el autómata cuando se encuentra en un estado determinado y recibe un símbolo específico.

Cada transición es leída desde el archivo de entrada en la forma:

$$\delta(I, X) = J \tag{2}$$

donde I representa el estado actual, X el símbolo leído y J el estado al que se debe transitar. Para registrar correctamente la transición dentro de la matriz, el programa primero busca la posición del símbolo dentro del arreglo del alfabeto. Una vez encontrado el índice correspondiente, la matriz se actualiza con el estado destino.

Este proceso permite construir una representación directa de la función de transición del autómata, lo que facilita el acceso a la información durante la simulación del sistema.

5.5 Lectura de las cadenas a evaluar

Después de definir completamente la estructura del autómata, el programa procede a leer las cadenas que deben ser analizadas. Cada una de estas cadenas está compuesta por símbolos pertenecientes al alfabeto definido previamente.

Las cadenas son almacenadas dentro de un arreglo de tipo *String*, cuyo tamaño corresponde al número de cadenas especificado en la entrada. De esta manera el programa puede procesarlas posteriormente de forma individual.

Durante la lectura también se verifica que el archivo contenga la cantidad correcta de cadenas. Si el archivo termina antes de tiempo o si falta alguna cadena por leer, el programa genera un mensaje de error para indicar que la entrada no cumple con el formato esperado.

5.6 Evaluación de las cadenas

Una vez que el autómata ha sido completamente definido, el programa inicia el proceso de evaluación de las cadenas. Para cada cadena se simula el comportamiento del autómata comenzando desde el estado inicial.

El proceso consiste en recorrer cada símbolo de la cadena de forma secuencial. Para cada símbolo leído, el programa determina su posición dentro del alfabeto y utiliza esa información para consultar la matriz de transiciones. De esta manera se obtiene el nuevo estado al que debe dirigirse el autómata.

Si durante este proceso se encuentra un símbolo que no pertenece al alfabeto, la cadena se rechaza inmediatamente. Esto se debe a que el autómata no tiene una transición definida para símbolos que no forman parte de su conjunto de entrada.

Una vez que se han procesado todos los símbolos de la cadena, el programa verifica si el estado final alcanzado pertenece al conjunto de estados de aceptación. Si el estado final es un estado de aceptación, el programa imprime el mensaje **ACEPTADA**. En caso contrario, se muestra el mensaje **RECHAZADA**.

5.7 Manejo de errores

El programa también incluye un bloque de control de excepciones que permite detectar posibles errores durante la lectura del archivo de entrada. Si ocurre algún problema al intentar acceder al archivo o si el formato de los datos no es el esperado, el sistema muestra un mensaje indicando que ocurrió un error al leer el archivo.

Este mecanismo evita que el programa se detenga de forma inesperada y proporciona información útil para identificar problemas en los datos de entrada.

5.8 Funcionamiento general del programa

En términos generales, el programa implementa una simulación directa de un Autómata Finito Determinista. Primero se construye la estructura del autómata a partir de la información proporcionada en el archivo de entrada. Posteriormente, cada cadena es procesada siguiendo las transiciones definidas hasta determinar si pertenece o no al lenguaje reconocido.

El uso de estructuras simples como arreglos y matrices permite representar de manera clara los diferentes componentes del autómata. Además, la simulación paso a paso del procesamiento de las cadenas facilita comprender cómo los modelos teóricos de la teoría de la computación pueden implementarse de forma práctica mediante un programa de computadora.

Este tipo de implementaciones resulta útil para reforzar la comprensión de los conceptos fundamentales relacionados con los autómatas finitos, ya que permite observar directamente cómo los estados y las transiciones determinan la aceptación o rechazo de una cadena dentro de un lenguaje formal. Adicionalmente, el código fuente se encuentra disponible en el repositorio de GitHub: Autómata Finito Determinista

6. Resultados

En esta sección se presentan los resultados obtenidos al ejecutar el programa desarrollado para la simulación de un Autómata Finito Determinista (AFD). El objetivo de las pruebas realizadas fue comprobar que el sistema implementado es capaz de interpretar correctamente la descripción del autómata, procesar las cadenas de entrada y determinar si estas son aceptadas o rechazadas de acuerdo con las reglas definidas por el modelo.

Las pruebas se realizaron utilizando un archivo de entrada con el formato especificado en la práctica, el cual contiene la definición completa del autómata, incluyendo los estados, el alfabeto, las funciones de transición, los estados de aceptación y las cadenas que deben ser evaluadas.

6.1 Definición del autómata utilizado en las pruebas

Para verificar el funcionamiento del programa se definió un autómata a través del archivo de entrada afd.txt. En este archivo se especificaron los parámetros que describen la estructura del AFD, incluyendo el número total de estados, el alfabeto utilizado, el conjunto de estados de aceptación y las funciones de transición.

Una vez leído el archivo, el programa construye internamente las estructuras necesarias para representar el autómata, permitiendo posteriormente simular su comportamiento al procesar las cadenas de entrada.

```
7 3 21 1 2 10
a b c
2 6
1 a 2
1 b 3
1 c 4
2 a 2
2 b 3
2 c 4
3 a 3
3 b 2
3 c 7
4 a 7
4 b 7
4 c 5
5 a 7
5 b 7
5 c 6
6 a 7
6 b 7
6 c 4
7 a 7
7 b 7
7 c 7
abcc
ababbbccc
abccc
abb
caabbccc
bbbbccc
bbbb
ccc
aaaacc
abbcccbab
```

Figure 1: Archivo de entrada utilizado para definir el autómata y las cadenas de prueba.

6.2 Representación del autómata

Antes de ejecutar el programa, el autómata también puede representarse de manera gráfica utilizando herramientas de simulación. Para esta práctica se utilizó el simulador `Automaton Simulator`, el cual permite visualizar los estados del autómata y las transiciones entre ellos.

Esta representación gráfica facilita la comprensión del comportamiento del autómata, ya que permite observar cómo se producen los cambios de estado al procesar los símbolos del alfabeto. Además, sirve como una referencia para comprobar que el comportamiento del programa coincide con el modelo teórico definido.

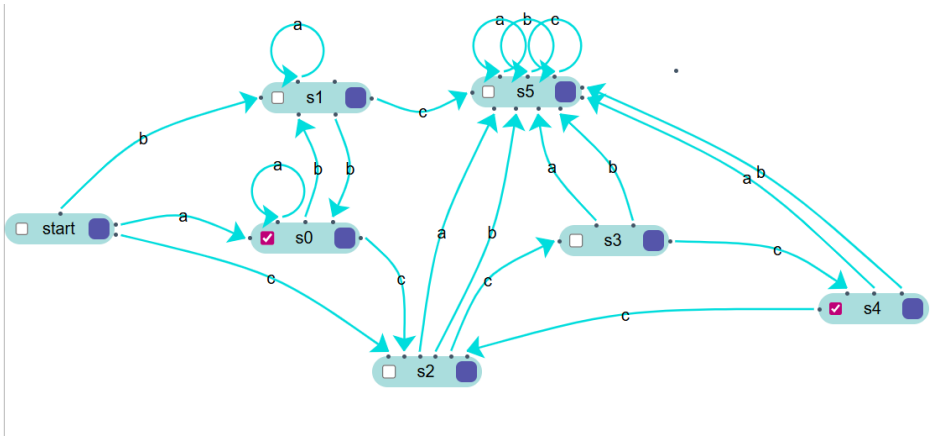

Figure 2: Representación gráfica del Autómata Finito Determinista utilizado en las pruebas.

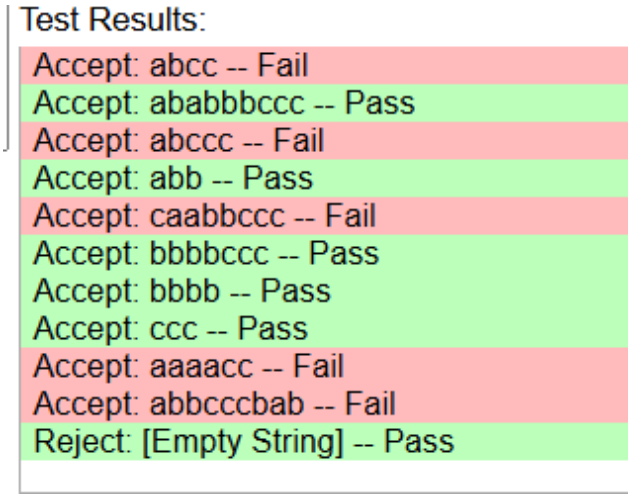

Figure 3: Resultados de simulación del Autómata Finito Determinista utilizado en las pruebas.

6.3 Ejecución del programa

Una vez definido el autómata y preparado el archivo de entrada, se ejecutó el programa desarrollado en *Apache NetBeans 17* utilizando el entorno de desarrollo configurado con *Java Development Kit 17*. Durante la ejecución, el sistema lee el archivo de entrada, construye la estructura interna del autómata y posteriormente analiza cada una de las cadenas proporcionadas.

El programa procesa cada cadena carácter por carácter, utilizando la matriz de transiciones para determinar el estado siguiente en cada paso. Al finalizar el procesamiento de cada cadena, el sistema verifica si el estado final pertenece al conjunto de estados de aceptación.

Si el estado final corresponde a un estado de aceptación, el programa imprime el mensaje `ACEPTADA`. En caso contrario, se muestra el mensaje `RECHAZADA`.

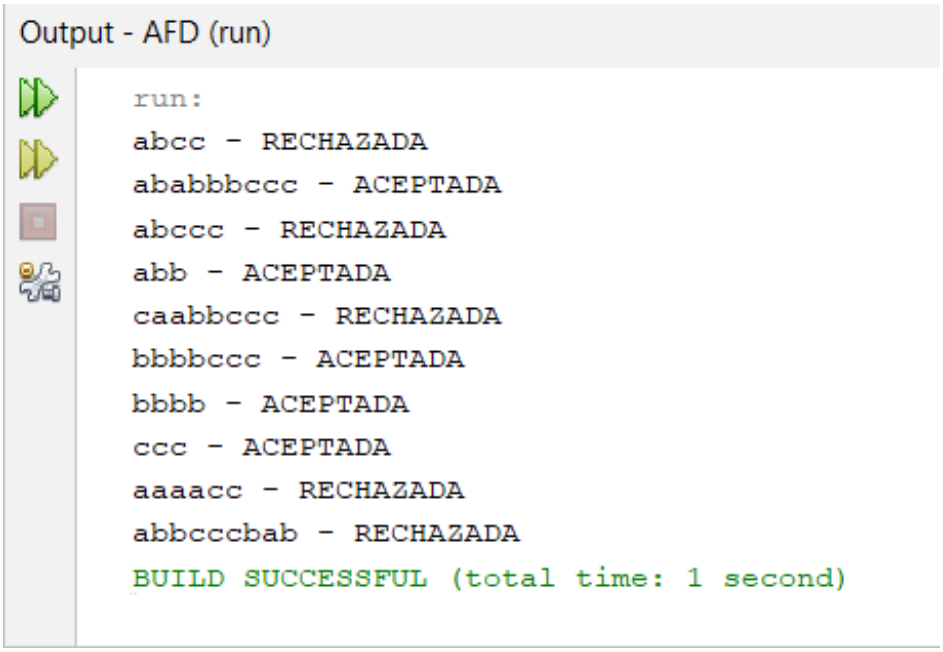

Figure 4: Resultado de la ejecución del programa mostrando la evaluación de las cadenas de entrada.

6.4 Análisis de los resultados

A partir de las pruebas realizadas se pudo comprobar que el programa implementa correctamente el comportamiento de un Autómata Finito Determinista. Las cadenas que cumplen con las reglas establecidas por el autómata fueron identificadas correctamente como aceptadas, mientras que aquellas que no siguen las transiciones definidas o que terminan en un estado no final fueron rechazadas.

El uso de estructuras de datos como arreglos y matrices permitió representar de manera eficiente los elementos del autómata, facilitando el acceso a las transiciones durante el procesamiento de las cadenas. Asimismo, las validaciones implementadas en el programa permitieron detectar errores en el formato de entrada o en los parámetros definidos.

En general, los resultados obtenidos demuestran que el programa es capaz de interpretar correctamente la descripción formal de un autómata y simular su funcionamiento para determinar la pertenencia de diferentes cadenas al lenguaje reconocido.

7. Cuestionario

Pregunta 1: ¿Qué paradigma de programación utilizaste para la implementación del AFD?, ¿Cuáles elementos del lenguaje de programación utilizaste?

Respuesta: Para la implementación del Autómata Finito Determinista se utilizó principalmente el paradigma de programación imperativo, ya que el programa sigue una secuencia de instrucciones que modifican el estado del sistema durante la ejecución. Este enfoque permite procesar cada símbolo de las cadenas de entrada de manera ordenada, actualizando el estado actual del autómata en cada paso.

El programa fue desarrollado en Java, utilizando elementos básicos del lenguaje como variables, arreglos, estructuras condicionales (if) y ciclos de iteración (for). Además, se emplearon clases como Scanner y File para la lectura de datos desde el archivo de entrada que contiene la definición del autómata y las cadenas a evaluar.

Pregunta 2: ¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?

Respuesta: Para representar los componentes del AFD se utilizaron principalmente arreglos y matrices. El alfabeto fue almacenado en un arreglo de caracteres, mientras que los estados de aceptación se representaron mediante un arreglo de tipo booleano.

Las funciones de transición se implementaron mediante una matriz bidimensional de enteros, donde las filas representan los estados del autómata y las columnas los símbolos del alfabeto. Finalmente, las cadenas a evaluar se almacenaron en un arreglo de tipo String, lo que permitió procesarlas posteriormente de manera individual.

Pregunta 3: ¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?

Respuesta: Las funciones de transición se implementaron utilizando una matriz bidimensional junto con ciclos for y estructuras condicionales. Estos elementos permiten recorrer cada símbolo de la cadena, identificar su posición dentro del alfabeto y determinar el siguiente estado del autómata.

Entre las limitaciones del programa se encuentra la dependencia de un formato específico del archivo de entrada, así como la falta de una interfaz gráfica, ya que los resultados se muestran únicamente en la consola. Además, el programa utiliza estructuras de datos simples, lo que limita su flexibilidad para representar autómatas más complejos.

Pregunta 4: ¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?

Respuesta: Para que el programa pueda evaluar correctamente un AFD es necesario que el archivo de entrada contenga todos los elementos que definen al autómata, incluyendo el número de estados, el alfabeto, las transiciones, el estado inicial y los estados de aceptación.

Asimismo, la función de transición debe estar correctamente definida para cada estado y cada símbolo del alfabeto. Finalmente, las cadenas que se evalúan deben estar formadas únicamente por símbolos que pertenezcan al alfabeto especificado, de lo contrario serán rechazadas por el programa.

8. Conclusiones

El desarrollo de esta práctica permitió comprender de manera más clara el funcionamiento de los Autómatas Finitos Deterministas y su aplicación dentro del área de la teoría de la computación. A través de la implementación de un programa fue posible observar cómo un modelo teórico puede representarse mediante estructuras de datos y algoritmos que simulan el procesamiento de cadenas de símbolos.

Durante la elaboración del programa se identificaron los componentes principales de un AFD, tales como el conjunto de estados, el alfabeto, la función de transición, el estado inicial y los estados de aceptación. Estos elementos fueron representados mediante arreglos y matrices dentro del programa, lo que permitió construir una estructura que reproduce el comportamiento del autómata al procesar diferentes cadenas de entrada.

Asimismo, la implementación del programa permitió comprobar de forma práctica el proceso mediante el cual un autómata analiza una cadena símbolo por símbolo, realizando transiciones entre estados hasta determinar si la cadena pertenece o no al lenguaje reconocido. Esto facilitó la comprensión de cómo los conceptos formales estudiados en la teoría pueden aplicarse en el desarrollo de soluciones computacionales.

Finalmente, la práctica también permitió reforzar el uso de herramientas de programación y el manejo de archivos de entrada para definir estructuras complejas. En conjunto, los resultados obtenidos demuestran que es posible implementar de manera efectiva un Autómata Finito Determinista mediante un programa sencillo, logrando simular su comportamiento y evaluar correctamente diferentes cadenas de símbolos.

Referencias Bibliográficas

References

[1] Hopcroft, J. E.; Motwani, R.; Ullman, J. D. (**2006**). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.

[2] Sipser, M. (**2012**). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

[3] Linz, P. (**2016**). *An Introduction to Formal Languages and Automata* (6th ed.). Jones & Bartlett Learning.

[4] Kozen, D. C. (**1997**). *Automata and Computability*. Springer.

[5] Sudkamp, T. A. (**2006**). *Languages and Machines: An Introduction to the Theory of Computer Science* (3rd ed.). Pearson.